Assignment – 1

ID5130: Parallel Scientific Computing

Indian Institute of Technology Madras

Shubham Jadhav

Roll No: CS24S009

Date: March 2026

Contents

1 Question 1: Triple Integral Evaluation (30 points) 2
1.1 Problem Statement . . . 2
1.2 Analytical Evaluation of the Triple Integral . . . 2
1.3 Part (a): Riemann Integration . . . 2
1.3.1 Approach . . . 2
1.3.2 Terminal Output . . . 3
1.3.3 Results . . . 3
1.3.4 Key Findings . . . 4
1.4 Part (b): Monte Carlo Integration . . . 4
1.4.1 Approach . . . 4
1.4.2 Terminal Output . . . 4
1.4.3 Results . . . 5
1.4.4 Plots . . . 5
1.4.5 Key Findings . . . 6

2 Question 2: Poisson Equation – Jacobi Method (70 points) 7
2.1 Problem Statement . . . 7
2.2 Verification of Exact Solution . . . 7
2.3 Parallelization Approach . . . 7
2.4 Part (a): Serial Jacobi Program . . . 7
2.4.1 Approach . . . 7
2.4.2 Terminal Output . . . 8
2.4.3 Results . . . 8
2.4.4 Plot: ϕ vs x at $y = 0.5$ . . . 8
2.4.5 Key Findings . . . 9
2.5 Part (b): OpenMP Parallel Jacobi Program . . . 9
2.6 Part (c): Verification and Performance vs Grid Size . . . 9
2.6.1 Serial vs Parallel Verification . . . 9
2.6.2 Terminal Output for $\Delta = 0.1, 0.01, 0.005$ . . . 10
2.6.3 Results . . . 10
2.6.4 Plot: Time vs Grid Size . . . 10
2.6.5 Key Findings . . . 11
2.7 Part (d): Performance vs Number of Threads ($\Delta = 0.005$) . . . 11
2.7.1 Terminal Output . . . 11
2.7.2 Results . . . 11
2.7.3 Plot: Time and Speedup vs Threads . . . 12
2.7.4 Key Findings . . . 12

3 Reference Material Used 12

Question 1: Triple Integral Evaluation (30 points)

Problem Statement

Evaluate the following integral using Riemann integration and Monte Carlo methods:

$$I = \int_{x=0}^{4} \int_{y=0}^{3} \int_{z=0}^{2} \left(4x^3 + xy^2 + 5y + yz + 6z\right)\, dz\, dy\, dx \tag{1}$$

Analytical Evaluation of the Triple Integral

We evaluate Eq. (1) by integrating from the innermost variable outward.

Step 1: Inner integral over z (limits 0 to 2):

$$\begin{aligned}\int_0^2 (4x^3 + xy^2 + 5y + yz + 6z)\, dz &= \left[4x^3 z + xy^2 z + 5yz + \frac{yz^2}{2} + 3z^2\right]_0^2 \\ &= 8x^3 + 2xy^2 + 10y + 2y + 12 \\ &= 8x^3 + 2xy^2 + 12y + 12\end{aligned}$$

Step 2: Middle integral over y (limits 0 to 3):

$$\begin{aligned}\int_0^3 (8x^3 + 2xy^2 + 12y + 12)\, dy &= \left[8x^3 y + \frac{2xy^3}{3} + 6y^2 + 12y\right]_0^3 \\ &= 24x^3 + 18x + 54 + 36 \\ &= 24x^3 + 18x + 90\end{aligned}$$

Step 3: Outer integral over x (limits 0 to 4):

$$\begin{aligned}\int_0^4 (24x^3 + 18x + 90)\, dx &= \left[6x^4 + 9x^2 + 90x\right]_0^4 \\ &= 6(256) + 9(16) + 90(4) \\ &= 1536 + 144 + 360 = \boxed{2040}\end{aligned}$$

Part (a): Riemann Integration

1.3.1 Approach

The triple integral is approximated using the midpoint Riemann sum:

$$I \approx \sum_{i=0}^{N_x-1} \sum_{j=0}^{N_y-1} \sum_{k=0}^{N_z-1} f\left(x_{i+\frac{1}{2}},\, y_{j+\frac{1}{2}},\, z_{k+\frac{1}{2}}\right) \Delta V \tag{2}$$

where $\Delta V = \Delta x \cdot \Delta y \cdot \Delta z$, and the sample points are at the midpoints of each sub-volume. The assignment specifies $N_x = 48$, $N_y = 36$, $N_z = 24$, giving $\Delta x = \Delta y = \Delta z = 1/12 \approx 0.08\overline{3}$ and a total of 41,472 evaluations.

Parallelization: The triple loop is parallelized using `#pragma omp parallel for collapse(3) reduction(+:result)`. The `collapse(3)` clause merges all three loops into a single iteration space for better load balancing, and `reduction` safely accumulates partial sums from each thread.

1.3.2 Terminal Output

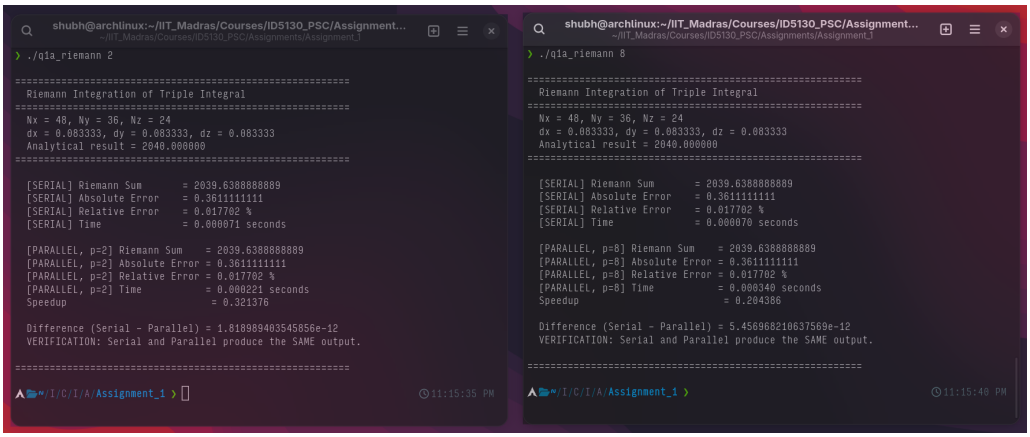


Figure 1: Q1(a): Terminal output of Riemann integration for $p = 2, 8$ threads.

1.3.3 Results

Table 1: Q1(a): Riemann Integration Results ($N_x = 48$, $N_y = 36$, $N_z = 24$)

Run	Result	Error (%)	Time (s)	Speedup
Serial	2039.6389	0.0177	0.000092	—
Parallel $p = 2$	2039.6389	0.0177	0.000267	0.35
Parallel $p = 4$	2039.6389	0.0177	0.000311	0.30
Parallel $p = 8$	2039.6389	0.0177	0.008633	0.01
Analytical	2040.0000	—	—	—

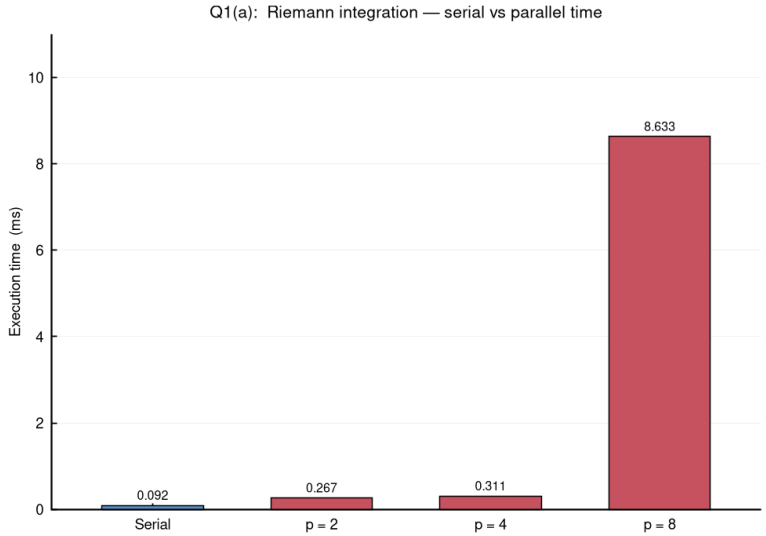


Figure 2: Q1(a): Serial vs parallel execution time for Riemann integration.

1.3.4 Key Findings

- The Riemann sum gives $I = 2039.6389$, which is within 0.018% of the analytical result $I = 2040$. The small error is due to the finite grid spacing.
- Serial and parallel programs produce **identical results** (difference $\sim 10^{-12}$), confirming correctness.
- Speedup is less than 1 for all thread counts because the problem size is very small (41,472 evaluations finishing in $\sim$0.09 ms). The OpenMP thread creation and synchronization overhead ($\sim$0.1–1 ms) completely dominates the computation. For larger problems, this overhead would be amortized.

Part (b): Monte Carlo Integration

1.4.1 Approach

The Monte Carlo method approximates the integral as:

$$I = V\bar{f} \pm V f_{\text{rms}}, \quad \bar{f} = \frac{1}{N}\sum_{l=1}^{N} f(x_l, y_l, z_l), \quad f_{\text{rms}} = \sqrt{\frac{\overline{f^2} - \bar{f}^2}{N}} \tag{3}$$

where $V = 4 \times 3 \times 2 = 24$ is the integration volume, and (x_l, y_l, z_l) are uniform random points. The error estimate $V f_{\text{rms}}$ decreases as $\mathcal{O}(1/\sqrt{N})$.

Parallelization: Each thread gets a unique seed via `seed = 42 + omp_get_thread_num() * 1000` using the thread-safe `rand_r()` function. The sums $\sum f$ and $\sum f^2$ use `reduction(+:...)` for safe accumulation.

1.4.2 Terminal Output

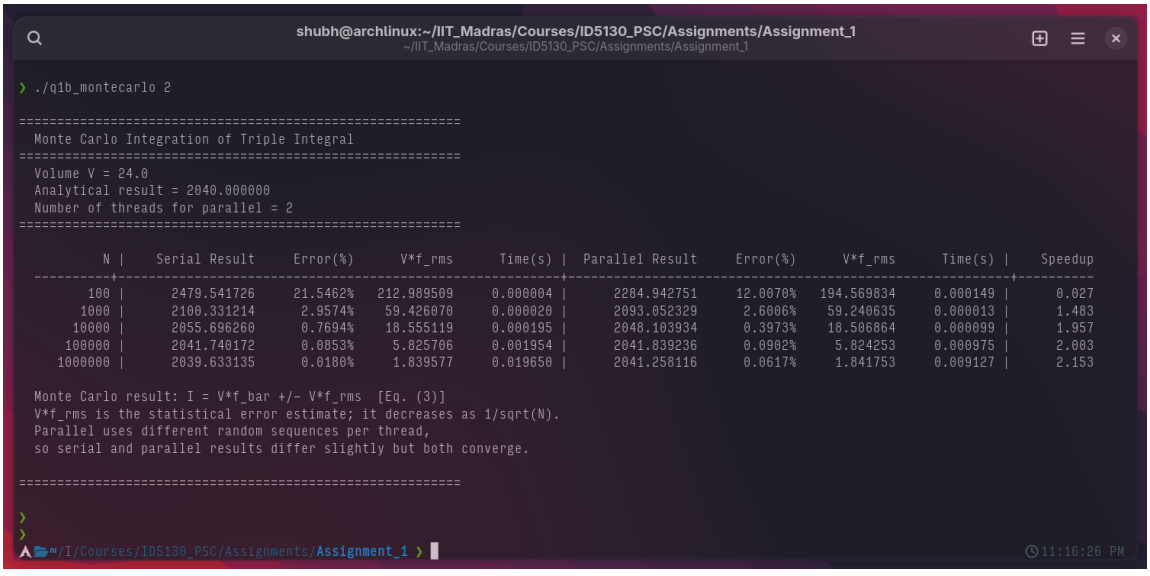


Figure 3: Q1(b): Terminal output of Monte Carlo integration for $p = 2$ threads.

1.4.3 Results

Table 2: Q1(b): Monte Carlo Serial Results and Error Estimates

N	**Result** ($V\bar{f}$)	**Error (%)**	Vf_{rms}	**Time (s)**
100	2479.54	21.55	212.99	0.000004
1,000	2100.33	2.96	59.43	0.000028
10,000	2055.70	0.77	18.56	0.000274
100,000	2041.74	0.09	5.83	0.002834
1,000,000	2039.63	0.02	1.84	0.028915

Table 3: Q1(b): Monte Carlo Parallel Execution Times (seconds)

	$p = 2$		$p = 4$		$p = 8$	
N	Time (s)	Speedup	Time (s)	Speedup	Time (s)	Speedup
100	0.000125	0.03	0.000168	0.02	0.000364	0.01
1,000	0.000020	1.41	0.000016	2.27	0.000011	3.49
10,000	0.000141	1.94	0.000108	3.42	0.000056	6.53
100,000	0.001409	2.01	0.001045	3.54	0.000523	7.08
1,000,000	0.014001	2.07	0.010318	3.35	0.005270	6.53

1.4.4 Plots

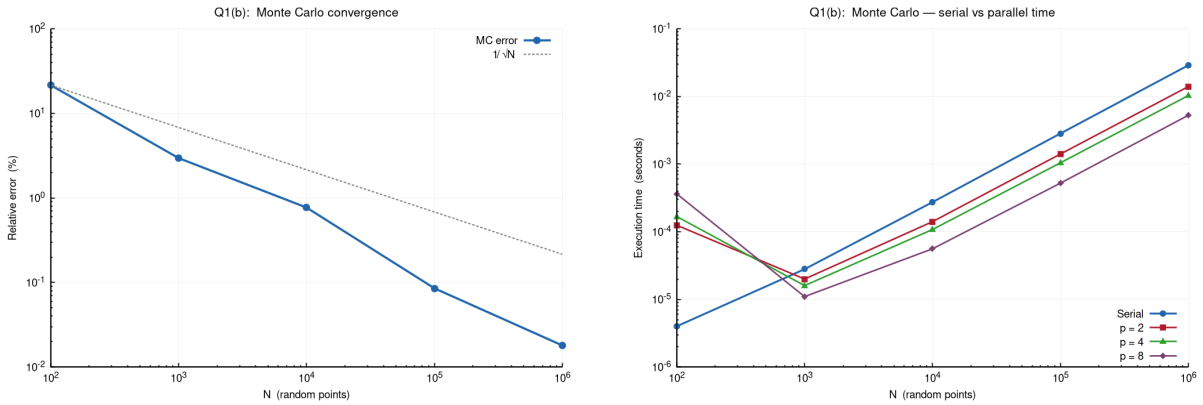


Figure 4: Q1(b): Monte Carlo convergence and execution time comparison.

1.4.5 Key Findings

- The error estimate Vf_{rms} decreases as $1/\sqrt{N}$: from $\sim$213 at $N = 100$ to $\sim$1.84 at $N = 10^6$ (a factor of $\sim$116, close to $\sqrt{10^4} = 100$). This confirms the theoretical convergence rate.
- For large N, Monte Carlo parallelizes very efficiently because it is **embarrassingly parallel** — each random evaluation is independent. At $N = 10^5$, speedup with $p = 8$ reaches 7.08×, nearly the ideal 8×.
- Parallel results differ slightly from serial because each thread uses a different random seed, but both converge to the same analytical value.
- For small N (100, 1000), the overhead of thread creation exceeds the computation, yielding speedup < 1.

Question 2: Poisson Equation – Jacobi Method (70 points)

Problem Statement

Solve the Poisson equation:

$$\nabla^2\phi = -q, \quad q = 2(2 - x^2 - y^2), \quad \phi(\pm 1, y) = 0, \quad \phi(x, \pm 1) = 0 \tag{4}$$

using the Jacobi iterative method. The discretized update equation is:

$$\phi_{i,j}^{(k+1)} = \frac{1}{4}\left[\phi_{i+1,j}^{(k)} + \phi_{i-1,j}^{(k)} + \phi_{i,j+1}^{(k)} + \phi_{i,j-1}^{(k)}\right] + \frac{\Delta^2}{4} q_{i,j} \tag{5}$$

The exact solution is $\phi(x, y) = (x^2 - 1)(y^2 - 1)$.

Verification of Exact Solution

We verify $\phi = (x^2 - 1)(y^2 - 1)$ satisfies Eq. (4):

$$\begin{aligned} \phi &= x^2y^2 - x^2 - y^2 + 1 \\ \frac{\partial^2\phi}{\partial x^2} &= 2y^2 - 2, \qquad \frac{\partial^2\phi}{\partial y^2} = 2x^2 - 2 \\ \nabla^2\phi &= 2x^2 + 2y^2 - 4 = -2(2 - x^2 - y^2) = -q \quad \checkmark \end{aligned}$$

Boundary conditions: $\phi(\pm 1, y) = (1 - 1)(y^2 - 1) = 0$ and $\phi(x, \pm 1) = (x^2 - 1)(1 - 1) = 0$. ✓

Parallelization Approach

The parallel Jacobi solver uses two key optimizations:

1. **Single parallel region:** Instead of creating/destroying threads at every iteration, one `#pragma omp parallel` region wraps the entire iteration loop. Inside, only `#pragma omp for` workshares the computation. This avoids fork/join overhead at each of the 10^5+ iterations.

2. **Parity-based double buffering:** Two arrays `phi[0]` and `phi[1]` alternate as read/write buffers using `rd = iter % 2` and `wr = 1 - rd`. This eliminates the need for a pointer swap inside `#pragma omp single` (and its extra barrier), leaving only one implicit barrier per iteration from `omp for`.

Part (a): Serial Jacobi Program

2.4.1 Approach

The serial Jacobi solver iterates Eq. (5) over all interior points ($1 \le i, j \le N - 1$) with $\Delta = 0.1$, giving a 21×21 grid (361 interior points). Initial guess is $\phi = 0$ everywhere. Convergence is checked every 100 iterations: the maximum relative error across all points must be $< 1\%$ of the exact solution.

2.4.2 Terminal Output

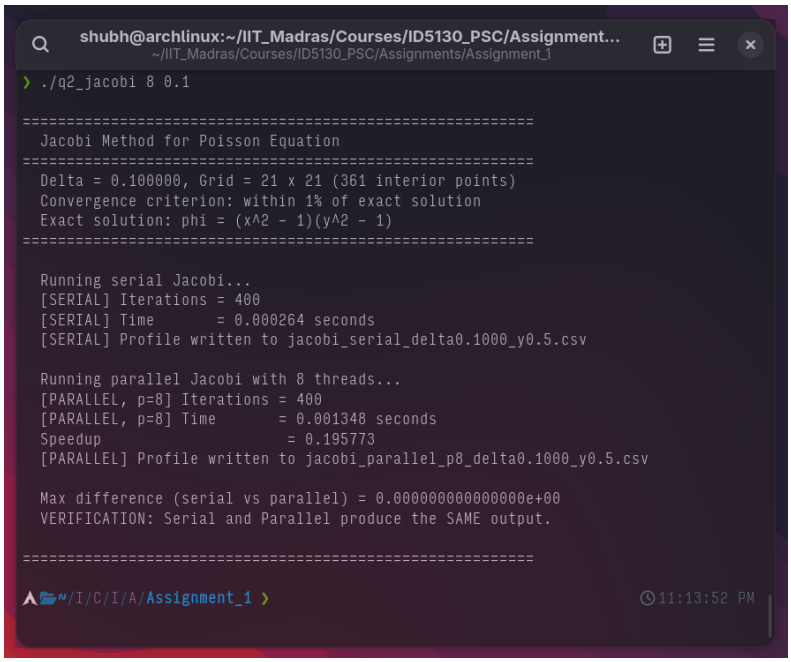


Figure 5: Q2(a)(b): Terminal output for Jacobi solver at $\Delta = 0.1$.

2.4.3 Results

- **Iterations to converge (within 1% of exact):** 400 iterations.
- **Serial time:** 0.000225 seconds (the grid is small).

2.4.4 Plot: ϕ vs x at $y = 0.5$

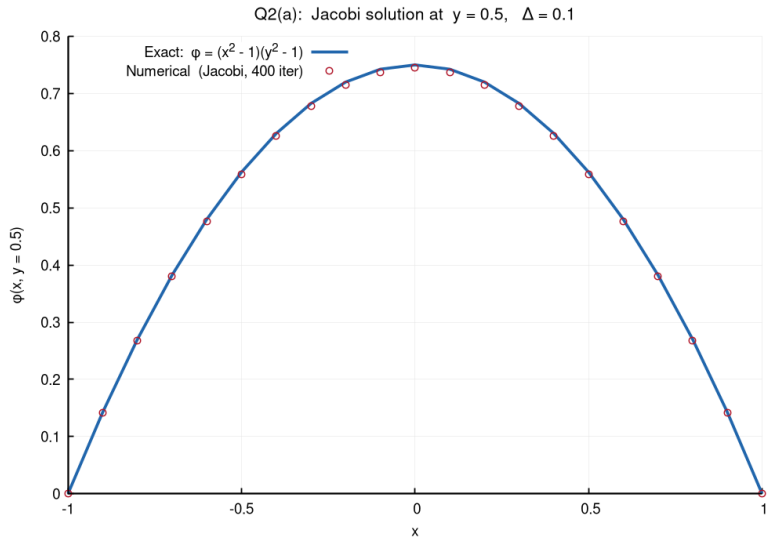


Figure 6: Q2(a): Numerical vs analytical solution of ϕ vs x at $y = 0.5$ for $\Delta = 0.1$.

2.4.5 Key Findings

- The numerical solution closely follows the analytical parabola $\phi(x, 0.5) = 0.75(1 - x^2)$, with a peak of $\sim$0.745 at $x = 0$ (exact: 0.75).
- The Jacobi method converges in 400 iterations for the 21×21 grid, which is consistent with the $\mathcal{O}(N^2)$ convergence behavior of Jacobi.

Part (b): OpenMP Parallel Jacobi Program

The parallel version uses the parity-based double buffering approach described above. The outer loop over rows (`i`) is parallelized with `#pragma omp for schedule(static)`, distributing rows evenly across threads. Since Jacobi uses only values from the previous iteration (no data dependencies within an iteration), this is inherently safe.

Part (c): Verification and Performance vs Grid Size

2.6.1 Serial vs Parallel Verification

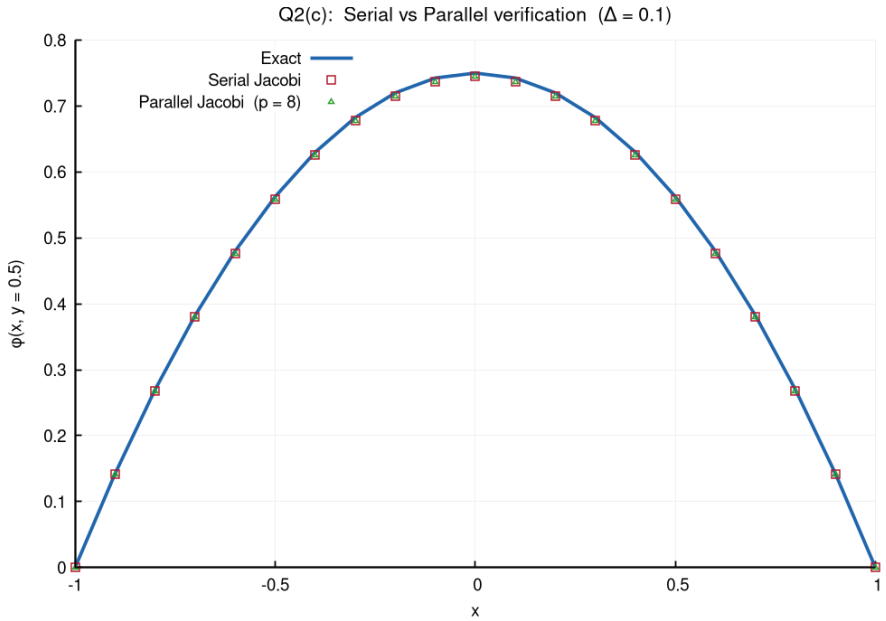


Figure 7: Q2(c): Serial vs Parallel Jacobi verification at $\Delta = 0.1$. Both overlap exactly.

The maximum pointwise difference between serial and parallel solutions is 0.0 (machine-zero), confirming the parallel program produces **identical results** to the serial version.

2.6.2 Terminal Output for $\Delta = 0.1, 0.01, 0.005$

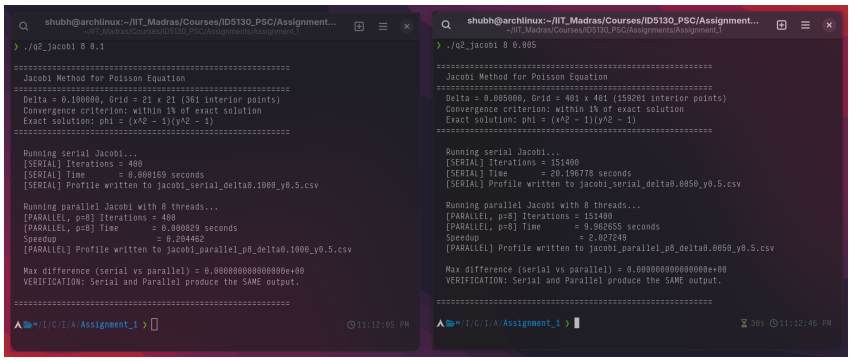


Figure 8: Q2(c): Terminal output for varying grid sizes with $p = 8$ threads.

2.6.3 Results

Table 4: Q2(c): Execution time vs grid size ($p = 8$ threads)

Δ	Grid	Iterations	Serial Time (s)	Parallel Time (s)	Speedup
0.1	21×21	400	0.000225	0.001050	0.21
0.01	201×201	37,900	2.256	0.685	3.29
0.005	401×401	151,400	33.827	9.615	3.52

2.6.4 Plot: Time vs Grid Size

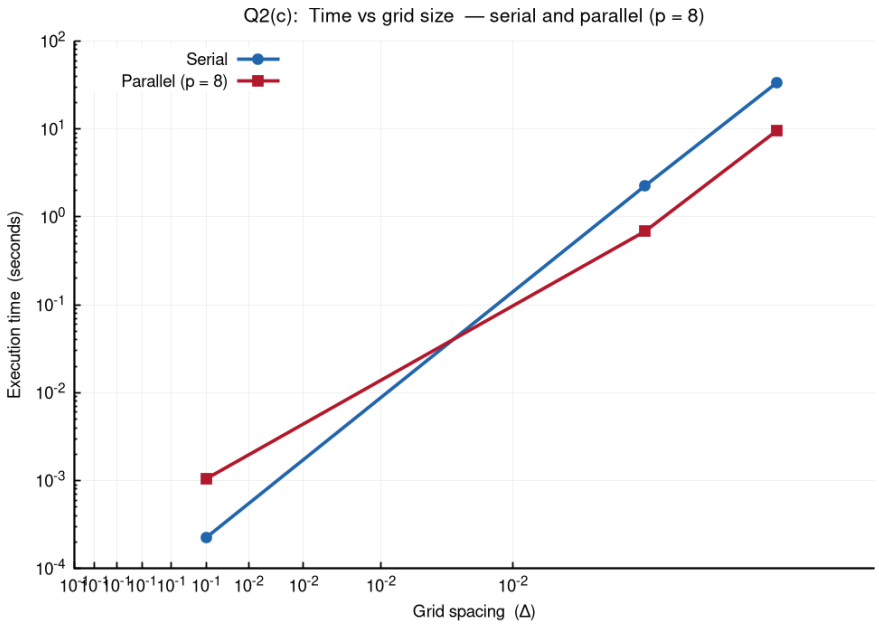


Figure 9: Q2(c): Execution time vs grid spacing for serial and parallel (8 threads).

2.6.5 Key Findings

- **Yes, there is clear performance improvement** for finer grids. At $\Delta = 0.1$ (361 interior points), the problem is too small for parallelism to help (speedup = 0.21). At $\Delta = 0.01$ and $\Delta = 0.005$, the speedup reaches 3.29× and 3.52×, respectively.
- The number of iterations grows roughly as $\mathcal{O}(1/\Delta^2)$: from 400 to 37,900 to 151,400. Combined with the per-iteration cost of $\mathcal{O}(N^2)$, the total work scales as $\mathcal{O}(1/\Delta^4)$, making parallel execution increasingly beneficial.
- The gap between serial and parallel widens with decreasing Δ (log-scale), confirming that OpenMP overhead is amortized for large problems.

Part (d): Performance vs Number of Threads ($\Delta = 0.005$)

2.7.1 Terminal Output

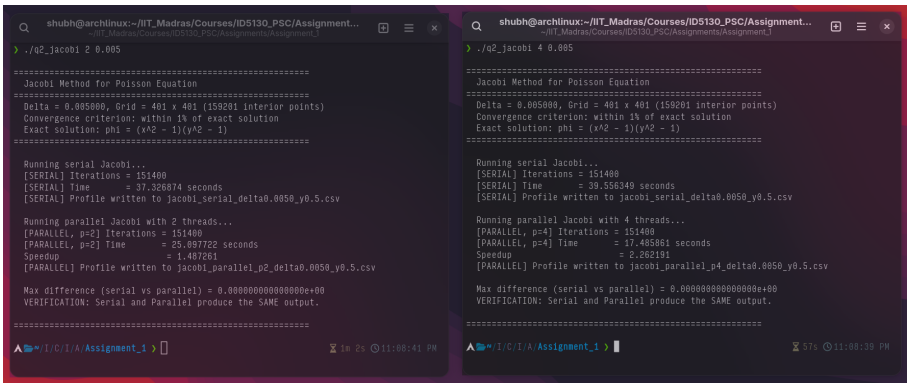


Figure 10: Q2(d): Terminal output for $\Delta = 0.005$ with $p = 2, 4$ threads.

2.7.2 Results

Table 5: Q2(d): Execution time vs threads for $\Delta = 0.005$ (401 × 401 grid, 151,400 iterations)

Threads (p)	Serial Time (s)	Parallel Time (s)	Speedup	Efficiency
1 (serial)	35.34	—	—	—
2	35.34	16.81	2.10	1.05
4	35.34	9.37	3.77	0.94
8	35.34	11.20	2.18	0.27
16	35.34	22.20	1.59	0.10

Note: Serial time averaged across runs (≈ 35.34 s). The $p = 8$ parallel time uses the re-run value of 11.20 s.

2.7.3 Plot: Time and Speedup vs Threads

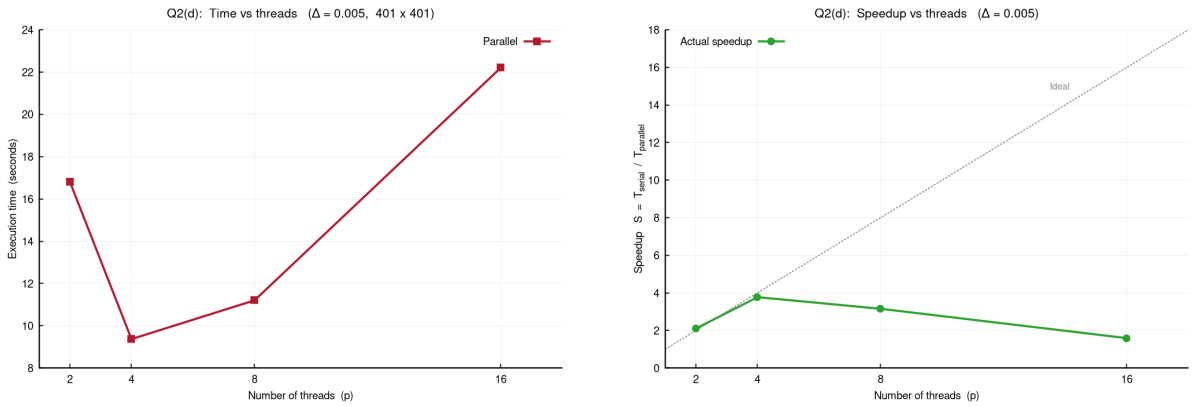


Figure 11: Q2(d): (Left) Execution time vs threads. (Right) Speedup vs threads with ideal reference.

2.7.4 Key Findings

- **Peak speedup occurs at $p = 4$ threads** (3.77×), which corresponds to the number of **physical cores** on the test machine (4 cores with hyperthreading → 8 logical cores).
- Beyond $p = 4$, performance **degrades**. At $p = 8$, each pair of hyperthreads shares the same physical core's ALU and L1/L2 cache. Since Jacobi is **memory-bandwidth bound** (each iteration reads and writes the entire 401×401 grid), the additional logical threads increase cache contention without adding compute capacity.
- At $p = 16$ (oversubscription: 16 threads on 8 logical cores), context-switching overhead further degrades performance to 1.59×.
- The efficiency (S/p) drops from near-ideal at $p = 2$ (1.05) to 0.10 at $p = 16$, illustrating Amdahl's law and hardware limitations for this memory-bound problem.

Reference Material Used

1. **Lecture Slides** – ID5130: Parallel Scientific Computing, Jan–May 2026, IIT Madras.
2. **OpenMP Official Documentation** – OpenMP Architecture Review Board. *OpenMP Application Programming Interface*, https://www.openmp.org.
3. **An Introduction to Parallel Programming** – Peter Pacheco, Morgan Kaufmann, 2011. ISBN: 978-0-12-374260-5.
4. **gnuplot** – T. Williams, C. Kelley, et al. *gnuplot – An Interactive Plotting Program*, http://www.gnuplot.info. Used for generating graphs and data visualization.